

Wave Data Analysis: A Microservice Application for Oceanographic Data

Tavishka Arora
Gabriel Bishop
Luke Venkataramanan

April 2025

Contents

1	Introduction	1
2	Data Description	2
2.1	Data Fields	2
2.2	Data Preprocessing and Cleaning	2
2.3	Dataset Scale	3
3	Key Technologies	4
3.1	Flask	4
3.2	Redis	4
3.3	Docker	4
3.4	Kubernetes	4
3.5	HotQueue	5
4	API Endpoints Overview	6
4.1	/	6
4.2	/help	6
4.3	/data	6
4.4	/waves?epoch={timestamp}	6
4.5	/jobs	6
4.6	/jobs/{job_id}	6
4.7	/results/{job_id}	7
4.8	/download/{job_id}	7
5	Usage Examples	8
6	Ethical and Professional Responsibilities	12
6.1	Data Integrity and Transparency	12
6.2	Accountability in Analysis and Interpretation	12
6.3	Security and Privacy Considerations	12
6.4	Environmental Stewardship and Responsible Use	12
6.5	Commitment to Professional Growth	12
7	Connection to Software Design Principles	13
7.1	Modularity: Cohesion and Low Coupling	13
7.2	Abstraction	13
7.3	Generalization	13
7.4	Portability and Reproducibility	13

1 Introduction

Oceanographic data collected from coastal buoys is critical in scientific and engineering fields. Measurements of wave height, peak period, sea surface temperature, and wave direction provide essential information for marine navigation, offshore construction, environmental monitoring, and climate modeling. However, working with real-world ocean data introduces several challenges. Large datasets must be stored efficiently, cleaned to remove invalid values, and made accessible to users ranging from researchers to developers. Traditional data handling methods, such as static spreadsheets or manual downloads, are insufficient for modern, scalable needs.

The objective of this project, **Wave Data Analysis**, is to design and implement a microservice-based web application that addresses these challenges through a containerized architecture. This application provides two main functionalities: (1) dynamic retrieval of wave measurement data, and (2) asynchronous processing of custom analytical jobs. Users can either query specific historical measurements based on time inputs or submit jobs that return summary statistics or graphical visualizations across selected months and years.

At its core, the application demonstrates how cloud-native software engineering techniques can be applied to scientific data workflows. By utilizing a modular design with distinct services for storage (Redis), web interaction (Flask), and background job processing (Worker), the project separates concerns across components while maintaining interoperability. Furthermore, the system is containerized using Docker, enabling consistent deployment across different machines, and it is configured for orchestration on Kubernetes clusters, showcasing production readiness.

The motivation for pursuing this architecture stems from the increasing demand for data-driven decision-making in fields like meteorology, oceanography, and civil engineering. Researchers and practitioners require rapid, reliable access to high-fidelity datasets without being burdened by backend infrastructure management. By automating data ingestion, analysis, and serving through a self-contained platform, **Wave Data Analysis** contributes a scalable model that could be adapted for numerous other real-world applications involving time series or geospatial data.

Although this project focuses specifically on buoy-collected wave measurements from Mooloolaba, Australia, the system design principles are broadly applicable. Future extensions could integrate predictive models, live buoy feeds, or machine learning-based anomaly detection, demonstrating how the architecture supports future growth and increasing data complexity.

2 Data Description

The dataset used in the **Wave Data Analysis** project originates from buoy-based oceanographic measurements collected near Mooloolaba, Australia. It was sourced from an openly available Kaggle repository titled “Waves Measuring Buoys Data (Mooloolaba).” This dataset captures wave characteristics over several years, offering valuable time-series information across multiple environmental variables.

The dataset can be found at the following link:

<https://www.kaggle.com/datasets/jolasa/waves-measuring-buoys-data-mooloolaba?>

2.1 Data Fields

Each record in the dataset corresponds to a timestamped observation and contains the following fields:

- **Date/Time:** The precise moment when the measurement was recorded, formatted as “MM/DD/YYYY HH:MM.”
- **Hs** (Significant Wave Height): Represents the mean wave height, in meters.
- **Hmax** (Maximum Wave Height): The highest single wave recorded during the measurement window, in meters.
- **Tz** (Zero-Crossing Period): The average time interval between successive zero-crossings of the water surface, in seconds.
- **Tp** (Peak Period): The period corresponding to the peak energy in the wave spectrum, in seconds.
- **Peak Direction:** The predominant direction from which the waves are approaching, relative to true North, in degrees.
- **SST** (Sea Surface Temperature): The temperature of the water surface at the time of recording, in °C.

Each measurement was generated by sophisticated buoy instrumentation designed to withstand marine conditions and deliver high-accuracy environmental readings. These measurements are vital for a variety of scientific and engineering applications, including modeling ocean currents, predicting coastal erosion, and ensuring the safety of maritime operations. It should be noted that we used the Pandas library for much of our data handling.

2.2 Data Preprocessing and Cleaning

Before the dataset could be reliably used within our application, it underwent critical preprocessing steps:

- **Handling Missing Values:** Observations where fields contained placeholder error values (such as 99.99 or -99.99) were replaced with NaN to indicate missing data.
- **Sorting Chronologically:** Records were sorted by their timestamp to facilitate efficient retrieval and time-based analyses. Since JSON objects are unordered, we sorted manually after getting the data.
- **Redis Storage Format:** After cleaning, each observation was stored in a Redis raw database with the **Date/Time** as the key and the remaining fields as the value, serialized as a JSON object.

The steps taken ensure that when users query the database, they receive clean, validated results. Moreover, this consistent data formatting was essential to enable the background worker system to operate reliably, whether generating statistics or producing graphical outputs.

2.3 Dataset Scale

The original dataset spans observations collected between January 2017 and December 2019. After preprocessing, it contained approximately 16,000 individual timestamped measurements. The manageable size of the dataset allowed for rapid prototyping during local development while still posing realistic challenges in terms of retrieval, analysis, and containerized deployment.

3 Key Technologies

The **Wave Data Analysis** project integrates several technologies from modern software development. Each tool was carefully selected for its ability to address specific architectural needs: lightweight web interaction, persistent in-memory data storage, asynchronous background processing, and scalable deployment.

3.1 Flask

Flask is a lightweight Python-based web framework used to build web applications quickly and flexibly. In this project, Flask serves as the backbone of the RESTful API, handling incoming HTTP requests, validating inputs, and returning responses. It allows users to interact with the system through endpoints such as `/data`, `/waves`, and `/jobs`, providing an easy-to-use interface for both data retrieval and job submission. Flask's simplicity and extensibility made it an ideal choice for rapidly developing and deploying a clean API without the overhead of larger full-stack frameworks like Django.

3.2 Redis

Redis is an open-source, in-memory data structure store that supports various types of abstract data structures such as strings, hashes, lists, and sets. It is widely used for caching, session management, and queuing systems. In this project, Redis plays three critical roles:

- Storing cleaned wave measurement data for efficient querying.
- Managing asynchronous job queues through a library called `HotQueue`.
- Persisting the results of completed analytical jobs for later retrieval by users.

Redis was chosen due to its extremely fast read/write operations, its simplicity, and its support for lightweight pub/sub mechanisms.

3.3 Docker

Docker is a platform for developing, shipping, and running applications inside containers. Containers package code and dependencies so that applications can run consistently in different computing environments. Each service in our system — the Flask API, Redis database, and Worker process — is encapsulated in its own container. This approach ensures that the application behaves the same way during local development, testing, and cloud deployment, dramatically reducing the “it works on my machine” problem.

3.4 Kubernetes

Kubernetes is an open-source system for automating the deployment, scaling, and management of containerized applications. For this project, Kubernetes allows us to orchestrate the Flask application, Redis database, and Worker service as distinct but interconnected deployments. Kubernetes manifests define how each container is deployed, how they communicate, and how storage persists across restarts. Key features leveraged include:

- **Deployments:** Ensure that the correct number of replicas are always running.
- **Persistent Volume Claims (PVCs):** Provide durable storage for Redis data.
- **Services:** Enable internal and external networking between containers.
- **Ingresses:** Allow external users to access the Flask API through a public endpoint.

The use of Kubernetes prepares the system for scaling to handle larger datasets or higher request volumes with minimal architectural changes.

The user can deploy the Kubernetes configuration files for either the test or production environments. In our project, we were instructed to make the two have no functional difference. Nominally, the test environment is used for developers to experiment and debug, while the prod environment is where real users interact with the application. For our purposes, they simply occupy different namespaces and are accessible at different URLs.

It should be noted that we used Docker Compose for testing our application on our local machine, and we used Kubernetes when deploying our application to the cluster on the TACC supercomputers. However, we still used the Docker images in our Kubernetes deployments.

If you want to run the URL routes found in section 5 with Kubernetes, you could replace the localhost URL with the host name for either the prod or test environments:

- Local (Docker Compose): `http://localhost:5000`
- Prod (k8s): `wave.coe332.tacc.cloud`
- Test (k8s): `wave-test.coe332.tacc.cloud`

Our application is configured so that anyone with access to the Internet can access our URL routes, since we use an NGINX ingress and a NodePort service, which allow external traffic to our deployments on the Kubernetes cluster.

3.5 HotQueue

HotQueue is a lightweight Python library that uses Redis lists to create a simple job queuing system, enabling concurrency and preventing race conditions. It provides decorators that allow background workers to listen for new jobs and pull them as they arrive. In **Wave Data Analysis**, HotQueue is used to manage the flow of analytical jobs, ensuring that the worker service processes them asynchronously without blocking the Flask application.

4 API Endpoints Overview

The **Wave Data Analysis** application exposes a set of RESTful API endpoints through its Flask server. These routes allow users to interact with the dataset, submit analytical jobs, retrieve results, and monitor system status. Each endpoint is designed to be intuitive, consistent, and informative.

Below is a detailed list of all available routes:

4.1 /

- **Method:** GET
- **Description:** Root endpoint returns “Hello, world!” message to confirm that the server is online.

4.2 /help

- **Method:** GET
- **Description:** Provides a human-readable description of all available endpoints and their expected usage.

4.3 /data

- **Methods:** POST, GET, DELETE
- **Description:** Manages the oceanographic dataset.
 - **POST:** Downloads and loads wave data from the remote Kaggle source if not already populated.
 - **GET:** Returns all currently stored wave data in JSON format.
 - **DELETE:** Clears the Redis database, removing all stored data.

4.4 /waves?epoch={timestamp}

- **Method:** GET
- **Description:** Retrieves the wave measurement record closest in time to a user-provided timestamp. Timestamps must be formatted as MM/DD/YYYY HH:MM.

4.5 /jobs

- **Methods:** POST, GET
- **Description:** Manages analytical job submissions.
 - **POST:** Submits a new job specifying a month, year, and method (**stats** or **plot**).
 - **GET:** Lists all existing job IDs and their statuses.

4.6 /jobs/{job_id}

- **Method:** GET
- **Description:** Retrieves detailed information for a specific job, including month, year, method, and status.

4.7 /results/{job_id}

- **Method:** GET
- **Description:** Retrieves the results of a completed job, including either a statistical summary or a graphical analysis.

4.8 /download/{job_id}

- **Method:** GET
- **Description:** Allows the user to download a generated histogram image showing wave height distributions for the specified month and year.

5 Usage Examples

This section demonstrates how users can interact with the **Wave Data Analysis** API. Each example shows a representative `curl` command and the expected output. These instructions are also found in more detail within the README.

Check if Server is Running

- **Command:**

```
curl http://localhost:5000/
```

- **Expected Output:**

```
Hello, world!
```

Retrieve Help Documentation

- **Command:**

```
curl http://localhost:5000/help
```

- **Expected Output:** List of all available API endpoints and usage instructions.

Load Dataset into Redis

- **Command:**

```
curl -X POST http://localhost:5000/data
```

- **Expected Output:**

```
201: Database has been successfully loaded with data.
```

Retrieve All Wave Data

- **Command:**

```
curl http://localhost:5000/data
```

- **Expected Output:**

```
{
  "09/12/2017 18:30": {
    "Hs": 1.318,
    "Hmax": 2.11,
    "Tz": 4.693,
    "Tp": 5.356,
```

```
    "Peak Direction": 23.0,  
    "SST": 20.95  
  }  
}
```

Delete All Wave Data

- Command:

```
curl -X DELETE http://localhost:5000/data
```

- Expected Output:

```
410: All data has been cleared from the database.
```

Query Closest Wave Record by Timestamp

- Command:

```
curl "http://localhost:5000/waves?epoch=09/12/2017%2018:30"
```

- Expected Output:

```
{  
  "Date/Time": "09/12/2017 18:30",  
  "Hs": 1.318,  
  "Hmax": 2.11,  
  "Tz": 4.693,  
  "Tp": 5.356,  
  "Peak Direction": 23.0,  
  "SST": 20.95  
}
```

Submit a New Job (Statistics Analysis)

- Command:

```
curl -X POST http://localhost:5000/jobs \  
-H "Content-Type: application/json" \  
-d '{"month":9,"year":2017,"method":"stats"}'
```

- Expected Output:

```
Job <job_id> successfully created. Status is Pending.
```

Submit a New Job (Plotting Analysis)

- **Command:**

```
curl -X POST http://localhost:5000/jobs \  
-H "Content-Type: application/json" \  
-d '{"month":9,"year":2017,"method":"plot"}'
```

- **Expected Output:**

```
Job <job_id> successfully created. Status is Pending.
```

List All Submitted Jobs

- **Command:**

```
curl http://localhost:5000/jobs
```

- **Expected Output:**

```
Here is a list of all current job IDs:  
Job 1. a1b2c3d4-e5f6-7890-abcd-1234efgh5678  
Job 2. z9y8x7w6-v5u4-3210-mnop-8765qrst4321
```

Get Details About a Specific Job

- **Command:**

```
curl http://localhost:5000/jobs/<job_id>
```

- **Expected Output:**

```
Here is all the information related to the job you requested:  
Job ID:      <job_id>  
Status:      Completed  
Month:       9  
Year:        2017  
Method:      Stats
```

Retrieve Results of a Completed Job

- **Command:**

```
curl http://localhost:5000/results/<job_id>
```

- **Expected Output:**

```
{  
  "Max Wave Height From Period Stats (m)": {...},  
  "Peak Direction From Period Stats (degrees)": {...},  
  "Sea Surface Temperature From Period Stats (degrees C)": {...}  
}
```

Download Histogram Plot from a Plotting Job

- **Command:**

```
curl -OJ http://localhost:5000/download/<job_id>
```

- **Expected Output:** The file `height_histogram.png` will be downloaded to the local machine.

6 Ethical and Professional Responsibilities

As engineers and developers, building data-driven applications comes with critical ethical and professional obligations. While the **Wave Data Analysis** project focuses on scientific oceanographic data rather than personal or sensitive information, important ethical considerations still apply.

6.1 Data Integrity and Transparency

It is our responsibility to ensure that any data made available through the application is accurate, complete, and clearly sourced. In this project, the wave measurements are derived from a Kaggle dataset openly provided for research and educational purposes. During preprocessing, special care was taken to properly clean the dataset (e.g., replacing invalid placeholder values with `NaN`) rather than discarding or altering data arbitrarily. Maintaining data integrity is essential so that future users — whether researchers or engineers — can trust the outputs they receive from the system.

6.2 Accountability in Analysis and Interpretation

Because the application supports user-driven statistical and graphical analysis, it is crucial that these analyses are implemented correctly and free from bias. Misrepresenting or oversimplifying environmental data could have serious real-world consequences, especially in fields like coastal engineering, environmental monitoring, or marine navigation. We approached all analysis methods (e.g., summary statistics, histogram plotting) with mathematical rigor and professional diligence, ensuring that the results accurately reflect the underlying data.

6.3 Security and Privacy Considerations

Although this particular application does not handle private user data, professional software design should still anticipate basic security principles. For example:

- Input validation is performed on all user-provided parameters to prevent injection attacks or server crashes.
- Services are containerized with minimal privileges, reducing potential security vulnerabilities in deployment.
- When scaling this architecture in the future (e.g., live buoy feeds), it would be essential to implement secure authentication, encrypted communication, and access controls.

6.4 Environmental Stewardship and Responsible Use

Oceanographic data represents a window into the health of marine ecosystems and the broader environment. As such, presenting this data responsibly and enabling meaningful analysis can contribute to a better understanding of climate patterns, sea-level changes, and habitat preservation. By making wave data more accessible and analyzable, our project reinforces the ethical commitment to supporting scientific inquiry and responsible environmental stewardship.

6.5 Commitment to Professional Growth

Finally, the **Wave Data Analysis** project serves as a model of professional software development practices, including modular design, containerization, version control, and team collaboration. Following such practices is not only a technical requirement but an ethical one: by building systems that are maintainable, scalable, and transparent, we ensure that future developers and users can benefit from our work without unnecessary technical debt or confusion.

7 Connection to Software Design Principles

The **Wave Data Analysis** project was intentionally structured to reflect important software design principles. These principles are crucial for building software that is robust, maintainable, extensible, and scalable.

7.1 Modularity: Cohesion and Low Coupling

The architecture of the application is highly modular. Each major functionality — web interaction, data storage, and asynchronous job processing — is handled by a separate component:

- **Flask API:** Handles all user interaction through clearly defined HTTP routes.
- **Redis Database:** Manages wave data storage, job metadata, and results independently of other services.
- **Worker Process:** Pulls jobs from a queue and executes background analyses asynchronously.

Within the codebase, intra-module cohesion is strong: files like `jobs.py` contain only job-related logic, and `worker.py` is focused exclusively on job execution. Meanwhile, inter-module coupling is minimized: for example, the Flask app interacts with Redis only through well-defined calls, and the Worker consumes jobs without needing to know the internal Flask logic. This separation ensures that components can be modified or replaced independently without breaking the entire system.

7.2 Abstraction

The project makes extensive use of abstraction to simplify complexity for users and developers. Instead of exposing users to the details of Redis storage commands or background queue mechanics, the application provides simple HTTP endpoints such as `/jobs` and `/results/{job_id}`.

Internally, complex tasks such as pulling wave records from Redis, parsing timestamps, and plotting histograms are encapsulated within Python functions and hidden behind clean interfaces. This mirrors the real-world use of abstractions in programming languages, such as Python’s file objects abstracting low-level file system operations.

7.3 Generalization

The application’s design supports generalization by allowing the same interface to support multiple analysis types (e.g., statistical summaries, histogram plots) without changing the way users interact with the system. Users simply specify a “method” parameter when submitting a job — no need to learn a new endpoint or data format for each analysis type.

Additionally, the Redis data storage approach and job queueing system are generalized enough that they could be reused for different datasets (e.g., weather data, seismic data) with minimal code modifications.

7.4 Portability and Reproducibility

From the start, the system was designed with portability in mind. All services (Flask app, Redis database, and Worker) are containerized using Docker, meaning they can be deployed identically across laptops, cloud servers, and Kubernetes clusters without manual environment setup. This approach greatly enhances portability across hardware and operating systems.

Reproducibility is also prioritized: datasets are loaded programmatically from remote sources, and analyses are deterministic based on user-provided parameters. As a result, given the same data snapshot and the same job request, the system will consistently produce the same results, fulfilling reproducibility goals.

Citations

- Kaggle dataset: *Waves Measuring Buoys Data (Mooloolaba)*, retrieved from <https://www.kaggle.com/datasets/jolasa/waves-measuring-buoys-data-mooloolaba>
- Flask Web Framework: Flask Documentation, <https://flask.palletsprojects.com/en/2.3.x/>
- Redis-Py Client: Redis-Py Documentation, <https://redis-py.readthedocs.io/en/stable/>
- HotQueue Python Library: HotQueue Documentation, <https://pypi.org/project/hotqueue/>
- Pandas Data Analysis Library: Pandas Documentation, <https://pandas.pydata.org/docs/>
- Matplotlib Plotting Library: Matplotlib Documentation, <https://matplotlib.org/stable/index.html>
- Requests HTTP Library: Requests Documentation, <https://requests.readthedocs.io/en/latest/>
- Docker Containerization Platform: Docker Documentation, <https://docs.docker.com/>
- Docker Compose Tool: Docker Compose Documentation, <https://docs.docker.com/compose/>
- Kubernetes Orchestration System: Kubernetes Documentation, <https://kubernetes.io/docs/>